



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ_____

КАФЕДРА _____СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ_____

Лабораторная работа №4
По курсу
«Методы машинного обучения в АСОИУ»
«Методы поиска условного экстремума»

Студент ИУ5-24М
(Группа)

(Подпись, дата) Сорокин А.Е.
(И.О.Фамилия)

Проверила

(Подпись, дата) Гапанюк Ю.Е.
(И.О.Фамилия)

Цель работы

- Изучение алгоритмов условной оптимизации.
- Разработка программ реализации алгоритмов условной оптимизации.
- Нахождение оптимальных решений для задач с учетом ограничений.

Задание

Требуется найти условный минимум тестовой функции Розенброка:

$$f(x) = \sum_{i=1}^{n-1} \left[a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2 \right] + f_0$$

Методом штрафных функций.

Методом барьерных функций.

Комбинированным методом штрафных и барьерных функций.

Методом модифицированных функций Лагранжа.

Методом проекции градиента

Вариант 8:

$a=158, b=2, f_0=40, n=2$

$$g_1(x_1, x_2) = x_1^2 + x_2^2 - 10 \leq 0$$

$$g_2(x_1, x_2) = -x_1 \leq 0$$

$$g_3(x_1, x_2) = -x_2 \leq 0$$

1. Найти все стационарные точки и значения функций, соответствующие этим точкам.
2. Сравнить точность и скорость сходимости получаемых решений различными методами условной оптимизации.
3. Реализовать алгоритмы программированием на одном из языков высокого уровня (C++, C#, Python, Haskell, и др.).
4. Отчет предоставить в стандартном виде (TEX, PDF).

Требования к отчету:

1. Отчет должен содержать:
 - 1.1. Титульный лист.
 - 1.2. Цель работы.
 - 1.3. Постановку задачи.

- 1.4. Проверку решения на допустимость.
2. Исследование выполнить с помощью написанной Вами программы с результатами в графическом виде.
3. Кроме текста исследования следует привести также текст исходного кода программ.
4. Отчет оформляется в формате PDF желательно в редакторе TEX.

Найти все стационарные точки и значения функций, соответствующие этим точкам.

$$f(x) = \sum_1^{n-1} \left[a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2 \right] + f_0$$

Сначала подставим значения параметров по варианту в формулу Розеброка и получим:

$$f(x) = \sum_1^{2-1} \left[158(x_i^2 - x_{i+1})^2 + 2(x_i - 1)^2 \right] + 40 = 158(x_1^2 - x_2)^2 + \\ + 2(x_1 - 1)^2 + 40$$

Ограничение:

$$g_1(x_1, x_2) = x_1^2 + x_2^2 - 10 \leq 0$$

Представляет из себя окружность с радиусом $\sqrt{10}$

Ограничения:

$$g_2(x_1, x_2) = -x_1 \leq 0 \rightarrow g_2(x_1, x_2) = x_1 \geq 0$$

$$g_3(x_1, x_2) = -x_2 \leq 0 \rightarrow g_3(x_1, x_2) = x_2 \geq 0$$

Выделяет первую четверть (квадрант) координатной плоскости.

То есть прошлое решение $x_1 = 1$ и $x_2 = 1$ не изменится, так как удовлетворяет ограничениям.

С прошлой лабораторной работы:

Вычислим градиент.

Найдем частную производную по первой переменной x_1 :

$$\frac{\partial f}{\partial x_1} = 4ax_1(x_1^2 - x_2) + 2b(x_1 - 1)$$

Теперь найдем частную производную по второй переменной x_2 :

$$\frac{\partial f}{\partial x_2} = -2a(x_1^2 - x_2)$$

Подставляем значения заданных параметров по варианту:

$$\frac{\partial f}{\partial x_1} = 632x_1(x_1^2 - x_2) + 4(x_1 - 1)$$

$$\frac{\partial f}{\partial x_2} = -316(x_1^2 - x_2)$$

Выражение:

$$\frac{\partial f}{\partial x_2} = -316(x_1^2 - x_2)$$

Принимает значение равное 0 только в случае, если: $(x_1^2 - x_2) = 0 \rightarrow x_2 = x_1^2$

Подставим это решение в первую частную производную:

$$\begin{aligned} \frac{\partial f}{\partial x_1} &= 632x_1(x_1^2 - x_2) + 4(x_1 - 1) = 632x_1(x_1^2 - x_1^2) + 4(x_1 - 1) = \\ &= 4(x_1 - 1) \rightarrow x_1 = 1 \end{aligned}$$

Получаем точку минимума $x_1 = 1$ и $x_2 = 1$

Подставляем найденные значения в исходную функцию:

$$f(x_1, x_2) = 158(1^2 - 1)^2 + 2(1 - 1)^2 + 40 = 40$$

Получается, что глобальный минимум исходной функции равен 40 в точке (1,1)

Проверим найденное решение посредством программирования различных алгоритмов: метод штрафных функций, метод барьерных функций, комбинированный метод штрафных и барьерных функций, метод комбинированных функций Лагранжа, метод проекции градиента.

Результаты выполнения алгоритмов:

```
Penalty [0.48569896 0.23263733] 40.53069661813932
Barrier [1.34073372 1.79897738] 40.23251326463836
Combined [0.51444156 0.26158669] 40.47301676948158
Augmented Lagrangian [0.89542877 0.8012288 ] 40.02192052248329
Projected Gradient [0.42294456 0.          ] 45.72179719420114
```

Как видно, все алгоритмы приблизились к глобальному минимуму для функции в точке 1,1 – это значение 40.

Метод Penalty оказался наименее точным.

Метод Barrier очень хорошо справился с поиском минимума функции.

Метод Combined справился лучше, чем Penalty.

Метод Augmented Lagrangian показал наилучший результат поиска минимума функции.

Видно, что метод Projected Gradient уперся в ограничение положительности для x_1 и x_2 .

Оценить скорость сходимости указанных алгоритмов и сравнить по времени получение результата оптимизации для разных методов.

Время выполнения:

	Метод	Минимум x	$f(x)$	Всего итераций	Время (сек)
0	Penalty	[0.6843, 0.4664]	40.199847	796	0.016539
1	Barrier	[1.316, 1.7331]	40.199971	804	0.017408
2	Combined	[0.6846, 0.4668]	40.199479	766	0.020763
3	Augmented Lagrangian	[0.6845, 0.4666]	40.199687	995	0.007604
4	Projected Gradient	[0.4229, 0.0]	45.721797	10000	0.112782

Поиск осуществлялся не до точного значения минимума функции 40, а до окрестности минимума функции с эпсилон равным 0.2.

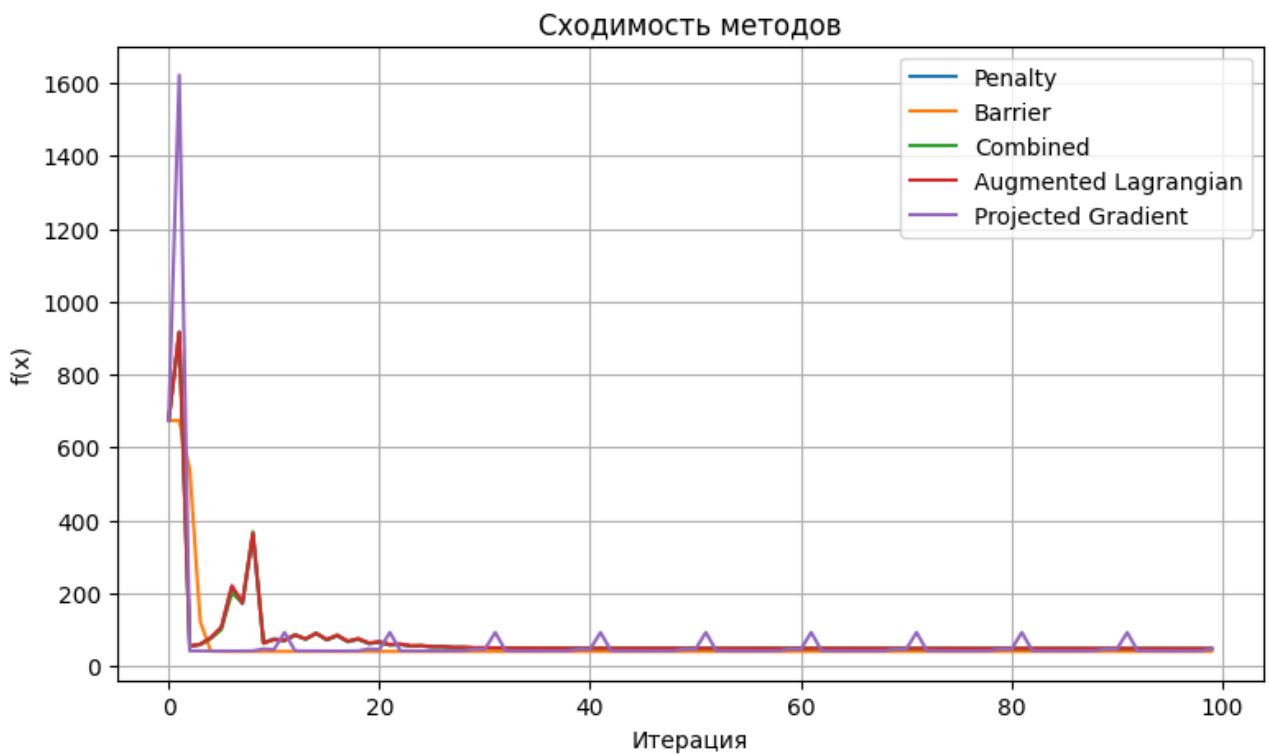


Рис. 1 – Графики сходимости различных методов к минимуму функции



Рис. 2 – График сходимости различных методов к минимуму функции с логарифмической шкалой для наглядности

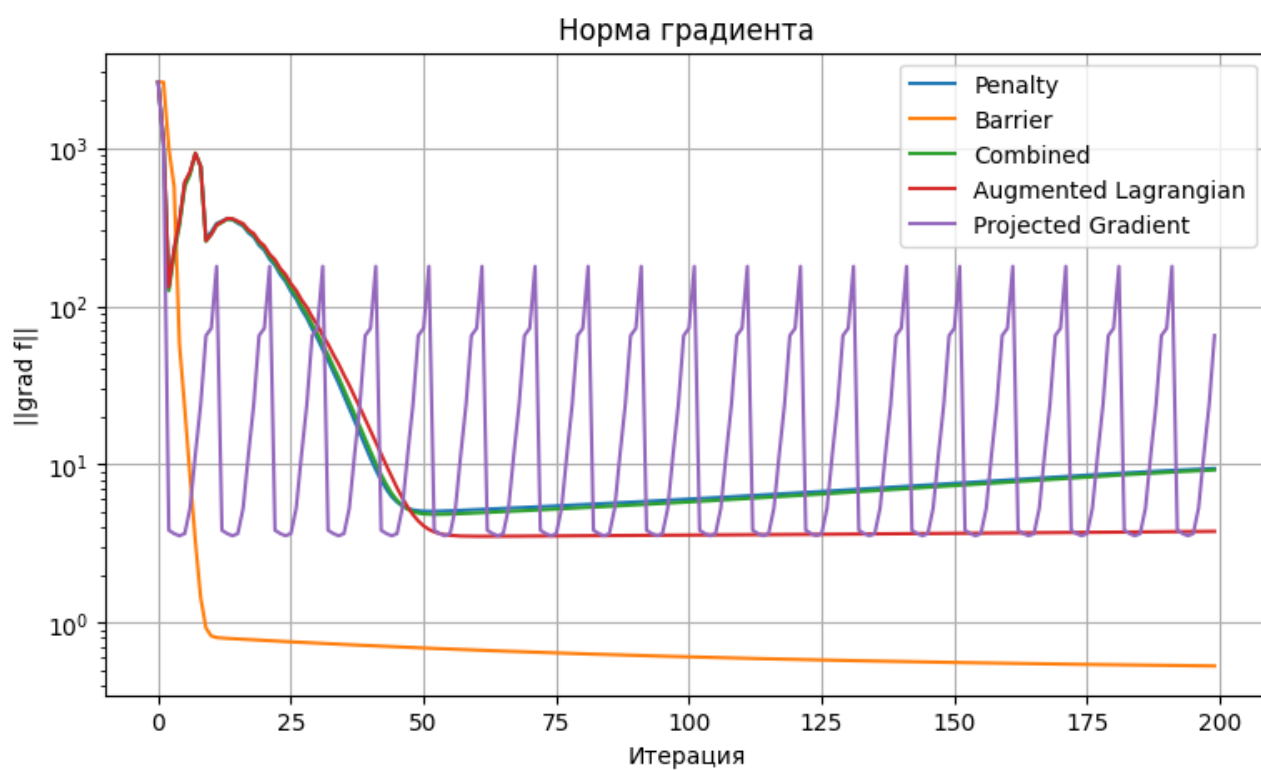


Рис. 3 – График изменения нормы градиента

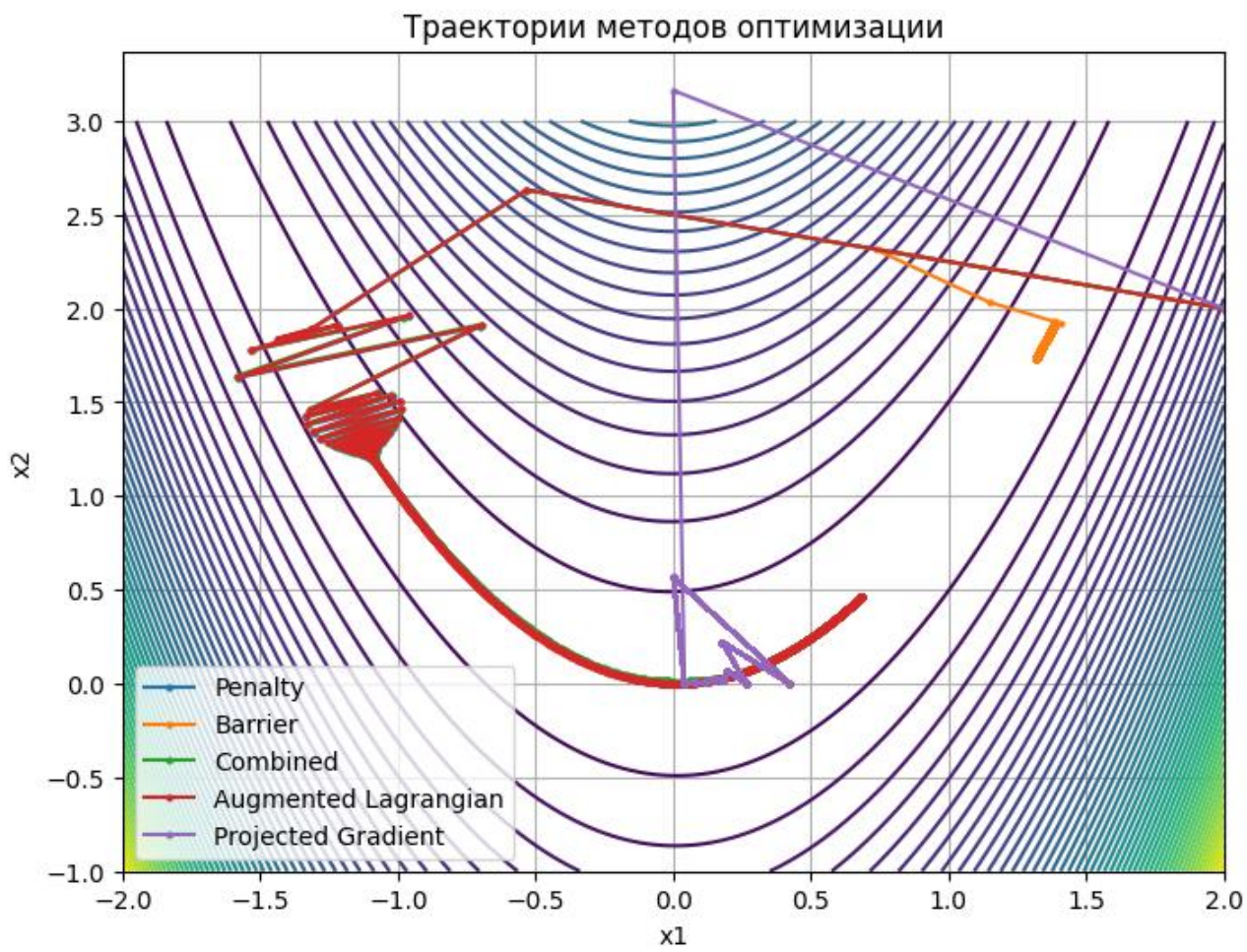


Рис. 4 – График с контуром исходной функции и спуск методов в точку минимума

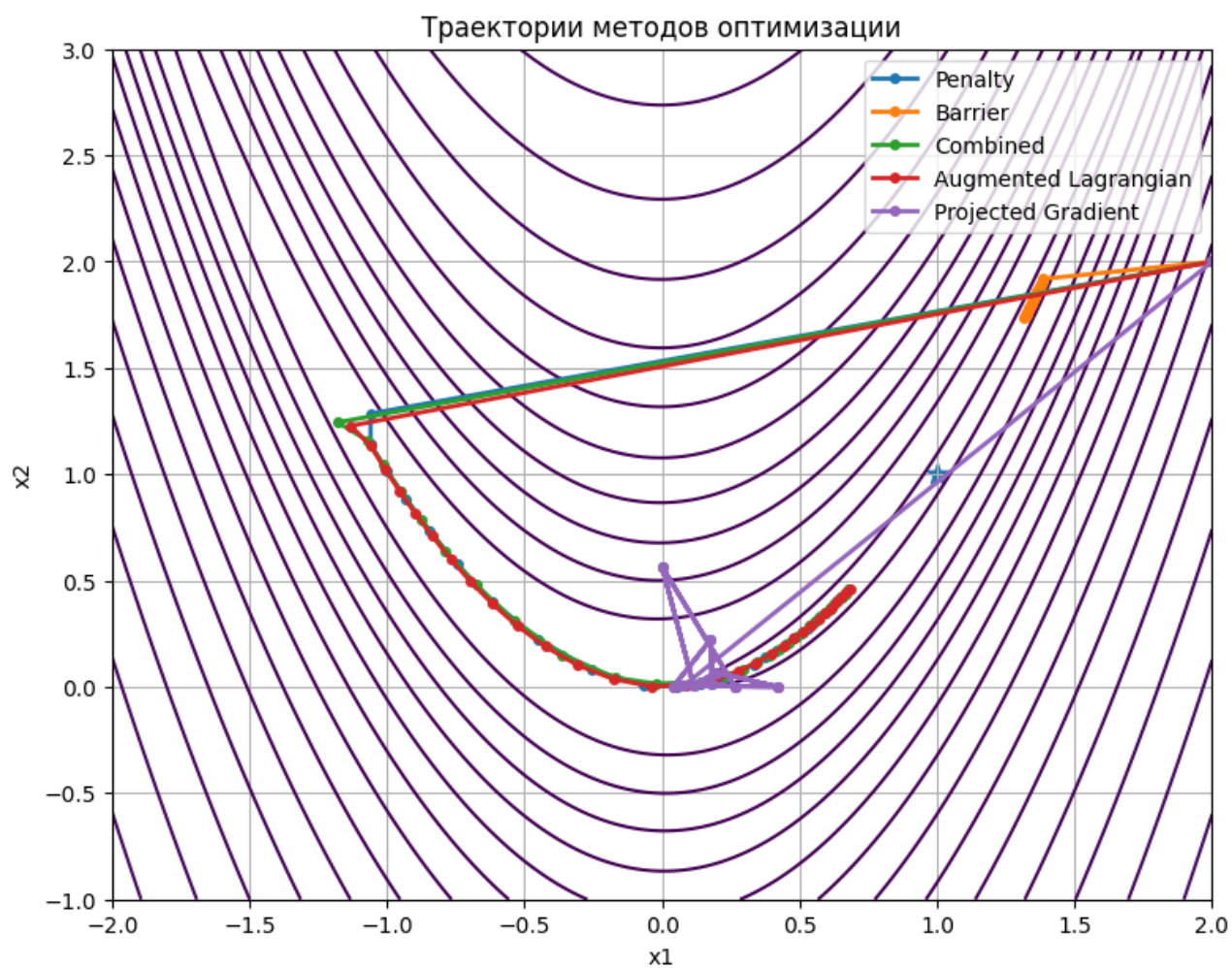


Рис. 5 – Тот же график с прореживанием точек

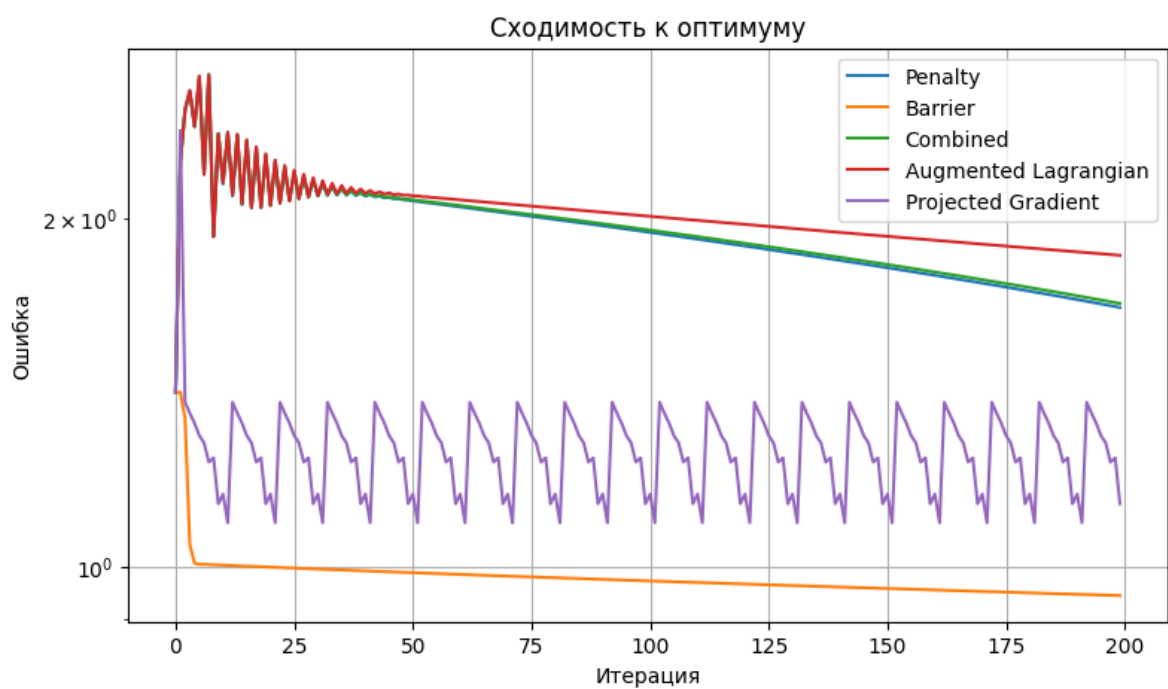


Рис. 6 – График сходимости к оптимуму – зависимость ошибки от количества итераций

Вывод: были выполнены все задачи по плану данной лабораторной работы. Соответствующие действия по выполнению задач лабораторной работы были описаны и внесены в настоящий отчет. Были изучены методы штрафных функций, барьерных функций, комбинированный метод штрафных и барьерных функций, модифицированных функций Лагранжа, проекции градиента. Была разработана программа реализации описанных методов многомерного поиска с ограничениями. Были вычислены условные экстремумы функции.

ПРИЛОЖЕНИЕ

Код программы с тестированием алгоритмов

```

import numpy as np

# Функция Розенброка
def f(x):
    x1, x2 = x
    return a*(x1**2 - x2)**2 + b*(x1 - 1)**2 + f0

# Функция расчета градиента
def grad(x):
    x1, x2 = x
    return np.array([
        4*a*x1*(x1**2 - x2) + 2*b*(x1 - 1),
        -2*a*(x1**2 - x2)
    ])

# Функция градиента для других методов
def numerical_grad(func, x, eps=1e-6):

    g = np.zeros_like(x)

    for i in range(len(x)):

        x1 = x.copy()
        x2 = x.copy()

        x1[i] += eps
        x2[i] -= eps

        g[i] = (func(x1) - func(x2)) / (2 * eps)

    return g

# ограничения
def g1(x): return x[0]**2 + x[1]**2 - 10
def g2(x): return -x[0]
def g3(x): return -x[1]

# Параметры и ограничения
a, b, f0 = 158, 2, 40
constraints = [g1, g2, g3]

```

Метод штрафных функций

Штрафуем за выход из допустимой области (четыре окружности с положительными значениями x_1 и x_2 с радиусом $\sqrt{10}$)

```
# Штрафная функция
def penalty_function(x, r):
    penalty = sum(max(0, g(x))**2 for g in constraints)
    return f(x) + r * penalty

# def penalty_method(x0, r=1, tol=1e-6):
#     x = x0.copy()

#     history = {
#         "x": [],
#         "f": [],
#         "grad_norm": []
#     }

#     for _ in range(10):
#         for _ in range(200):

#             history["x"].append(x.copy())
#             history["f"].append(f(x))
#             history["grad_norm"].append(np.linalg.norm(grad(x)))

#             g = grad(x)
#             x = x - 0.001 * g # градиентный шаг равен 0.001

#         r *= 10 # усиливаем штраф

#     return x, history

def penalty_method(x0, r=1, lr=0.001, max_iter=796):

    x = x0.copy()

    history = {
        "x": [],
        "f": [],
```

```

    "grad_norm": []
}

for _ in range(max_iter):

    history["x"].append(x.copy())
    history["f"].append(f(x))

    grad_p = numerical_grad(
        lambda z: penalty_function(z, r),
        x
    )

    history["grad_norm"].append(np.linalg.norm(grad_p))

    x = x - lr * grad_p

    r *= 1.01

return x, history

```

Метод барьерных функций

Не даем значениям выходить за ограничения. Поиск только внутри допустимой области.

Все просто - если выходим за ограничения, то возвращаем бесконечно большое значение

```

def barrier_function(x, mu):
    barrier = 0
    for g in constraints:
        val = g(x)
        if val >= 0:
            # return np.inf
            return 1e10
        barrier += np.log(-val)
    return f(x) - mu * barrier

```

Проверка допустимости градиентов, чтобы не выйти за ограничения, что в данном методе является непроходимым "барьером"

```

def feasible(x):

    return all(g(x) < 0 for g in constraints)

```

```

# def barrier_method(x0, mu=1, tol=1e-6):
#     x = x0.copy()

#     history = {
#         "x": [],
#         "f": [],
#         "grad_norm": []
#     }

#     for _ in range(10):
#         for _ in range(200):

#             history["x"].append(x.copy())
#             history["f"].append(f(x))
#             history["grad_norm"].append(np.linalg.norm(grad(x)))

#             g = grad(x)
#             x = x - 0.001 * g

#         mu *= 0.5

#     return x, history

```

```

def barrier_method(x0, mu=1, lr=0.001, max_iter=804):

    x = x0.copy()

    history = {
        "x": [],
        "f": [],
        "grad_norm": []
    }

    for _ in range(max_iter):

        history["x"].append(x.copy())
        history["f"].append(f(x))

        grad_b = numerical_grad(
            lambda z: barrier_function(z, mu),

```



```

        x
    )

    history["grad_norm"].append(np.linalg.norm(grad_b))

    x_new = x - lr * grad_b

    if feasible(x_new):
        x = x_new
    else:
        lr *= 0.5

    mu *= 0.99

    return x, history

```

Комбинированный метод

Внутри используем барьер

Снаружи штраф

```

def combined_function(x, r, mu):
    penalty = sum(max(0, g(x))**2 for g in constraints)

    barrier = 0
    for g in constraints:
        if g(x) < 0:
            barrier += np.log(-g(x))

    return f(x) + r*penalty - mu*barrier

```

```

# def combined_method(x0):
#     x = x0.copy()
#     r, mu = 1, 1

#     history = {
#         "x": [],
#         "f": [],

```

```

#         "grad_norm": []
#     }

#     for _ in range(10):
#         for _ in range(200):

#             history["x"].append(x.copy())
#             history["f"].append(f(x))
#             history["grad_norm"].append(np.linalg.norm(grad(x)))

#             g = grad(x)
#             x = x - 0.001 * g

#         r *= 10
#         mu *= 0.5

#     return x, history

```

```

def combined_method(x0, r=1, mu=1, lr=0.001, max_iter=766):

```

```

    x = x0.copy()

```

```

    history = {
        "x": [],
        "f": [],
        "grad_norm": []
    }

```

```

    for _ in range(max_iter):

```

```

        history["x"].append(x.copy())
        history["f"].append(f(x))

```

```

        grad_c = numerical_grad(
            lambda z: combined_function(z, r, mu),
            x
        )

```

```

        history["grad_norm"].append(np.linalg.norm(grad_c))

```

```

        x = x - lr * grad_c

```

```

    r *= 1.01
    mu *= 0.99

    return x, history

```

Метод модифицированных функций Лагранжа

$$L(x, \lambda^k, \mu^k, r^k) = f(x) + \sum_{j=1}^l \lambda_j^k g_j(x) + \frac{r^k}{2} \sum_{j=1}^l g_j(x)^2 + \frac{1}{2r^k} \sum_{j=l+1}^m \{[\max(0, \mu_j^k + r^k g_j(x))]^2 - (\mu_j^k)^2\}$$

```

def augmented_lagrangian(x0):
    x = x0.copy()
    lambdas = np.zeros(3)
    r = 1

    history = {
        "x": [],
        "f": [],
        "grad_norm": []
    }

    for _ in range(5):
        for _ in range(199):

            history["x"].append(x.copy())
            history["f"].append(f(x))
            history["grad_norm"].append(np.linalg.norm(grad(x)))

            g = grad(x)
            x = x - 0.001 * g

        for i, g_fun in enumerate(constraints):
            lambdas[i] += r * g_fun(x)

        r *= 2

    return x, history

```

Метод проекции градиента

шаг градиента + проекция в допустимую область

```
def project(x):
    # x1 >= 0, x2 >= 0
    x = np.maximum(x, 0)

    # круг
    norm = np.linalg.norm(x)
    if norm > np.sqrt(10):
        x = x / norm * np.sqrt(10)

    return x

def projected_gradient(x0, lr=0.01, max_iter=10000):
    x = x0.copy()

    history = {
        "x": [],
        "f": [],
        "grad_norm": []
    }

    for _ in range(max_iter):

        history["x"].append(x.copy())
        history["f"].append(f(x))
        history["grad_norm"].append(np.linalg.norm(grad(x)))

        x = x - lr * grad(x)
        x = project(x)

    return x, history
```

Запуск всех методов

```
x0 = np.array([2.0, 2.0])
```

```

methods = {
    "Penalty": penalty_method,
    "Barrier": barrier_method,
    "Combined": combined_method,
    "Augmented Lagrangian": augmented_lagrangian,
    "Projected Gradient": projected_gradient
}

for name, method in methods.items():
    x_res, history = method(x0)
    print(name, x_res, f(x_res))

Penalty [0.68434754 0.46642473] 40.19984743601045
Barrier [1.31598442 1.7331431 ] 40.19997099764037
Combined [0.68463876 0.46682553] 40.199478632618074
Augmented Lagrangian [0.68447408 0.46659881] 40.19968717163583
Projected Gradient [0.42294456 0.          ] 45.72179719420114

```

Вывод

Метод Penalty оказался наименее точным

Метод Barrier очень хорошо справился с поиском минимума функции

Метод Combined справился лучше, чем Penalty

Метод Augmented Lagrangian показал наилучший результат поиска минимума функции

Видно, что метод Projected Gradient уперся в ограничение положительности для x_1 и x_2

График сходимости функции

```

import matplotlib.pyplot as plt
plt.figure(figsize=(9,5))

```

```

max_points = 100

```

```

for name, method in methods.items():

```

```

    _, history = method(x0)

```

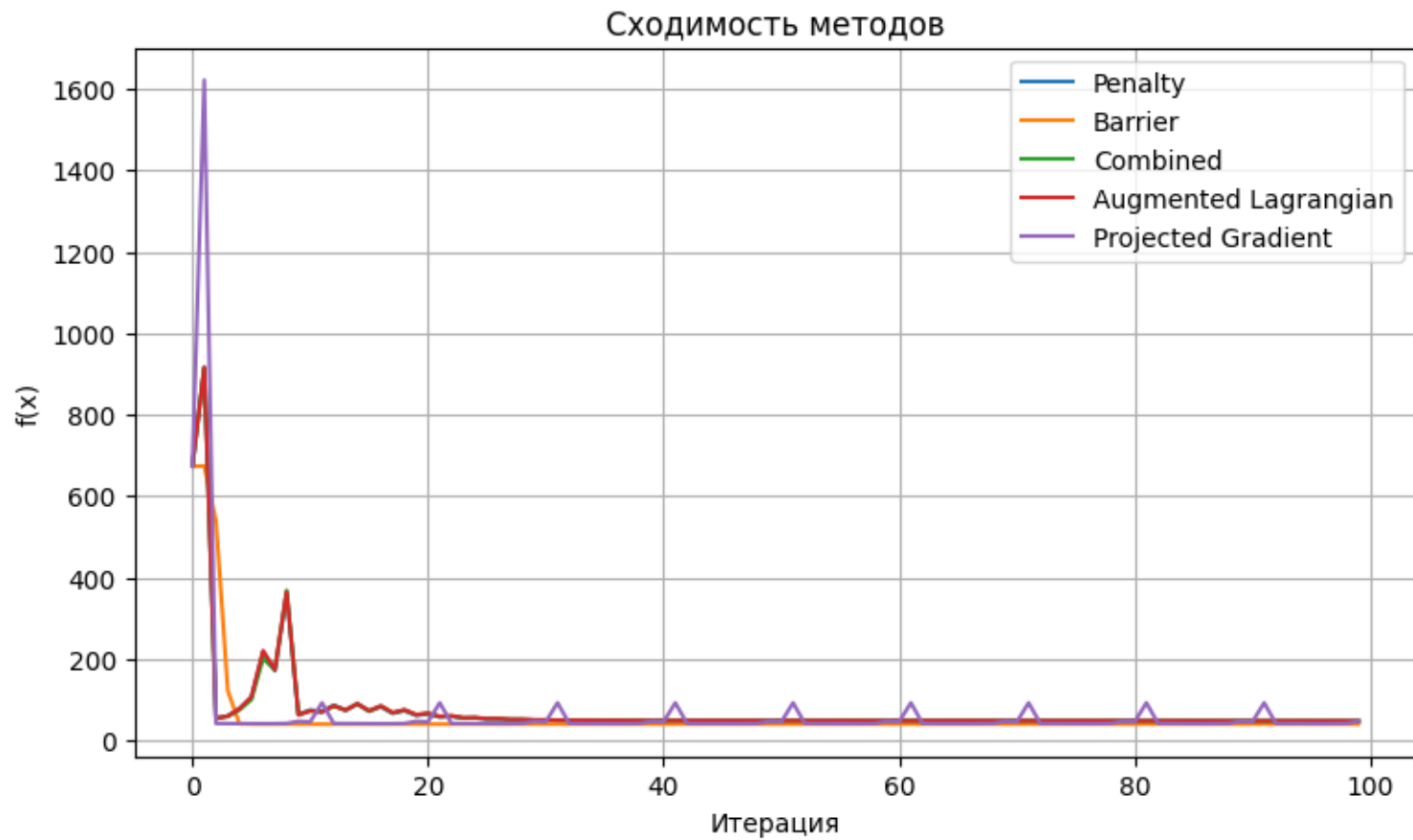
```

values = history["f"][:max_points]

plt.plot(values, label=name)

plt.xlabel("Итерация")
plt.ylabel("f(x) ")
plt.title("Сходимость методов")
plt.legend()
plt.grid()
plt.show()

```



```
plt.figure(figsize=(9,5))
```

```
max_points = 100

for name, method in methods.items():
    _, history = method(x0)
    values = history["f"][:max_points]
    plt.plot(values, label=name)

plt.yscale("log")

plt.xlabel("Итерация")
plt.ylabel("log(f(x))")
plt.title("Сходимость методов (логарифмическая шкала)")
plt.legend()
plt.grid()
plt.show()
```



График нормы градиента

Как изменяется градиент по мере выполнения алгоритма

```
plt.figure(figsize=(9,5))

max_points = 200

for name, method in methods.items():
    _, history = method(x0)
```



```

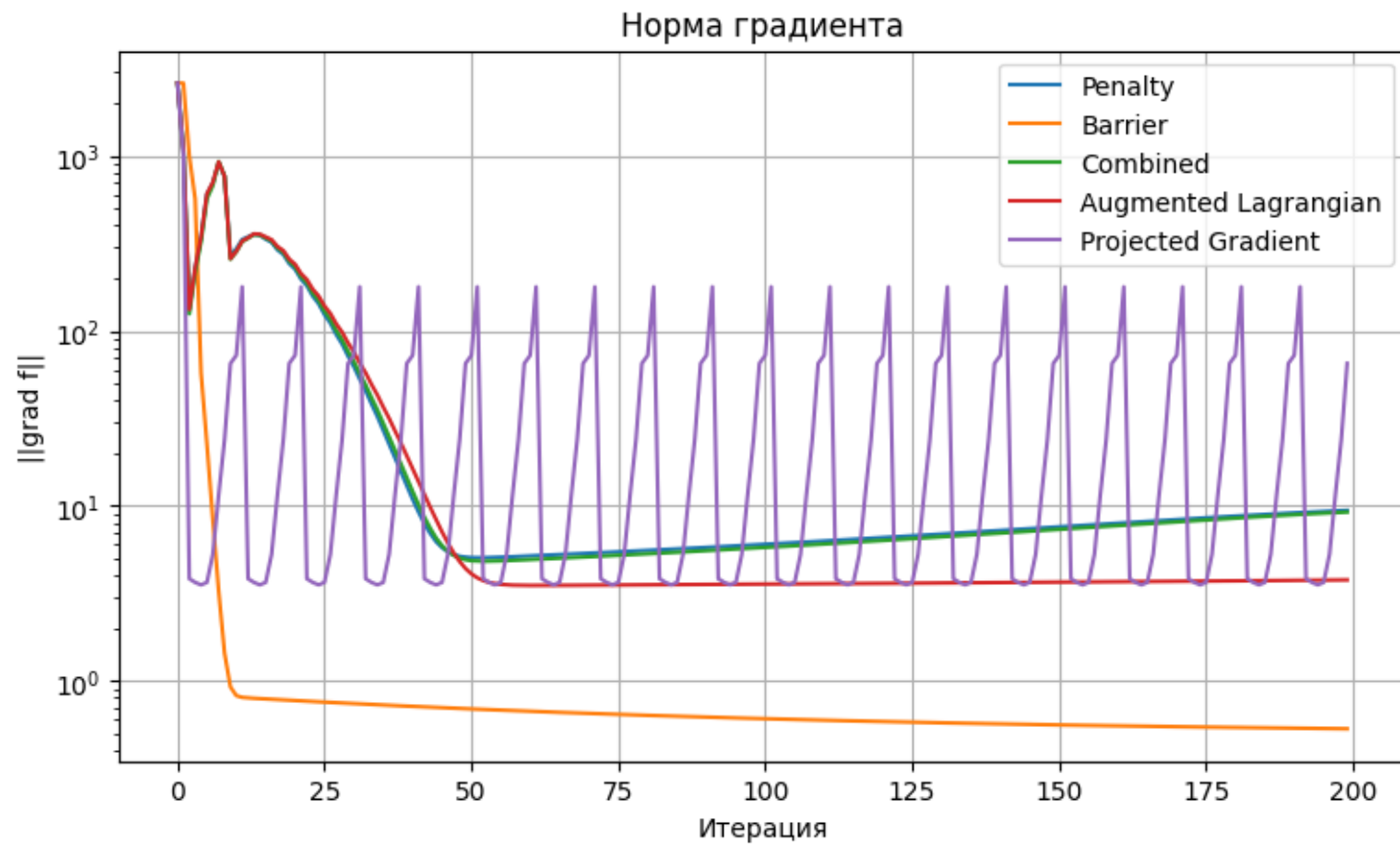
grads = history["grad_norm"][:max_points]

plt.plot(grads, label=name)

plt.yscale("log")

plt.xlabel("Итерация")
plt.ylabel("||grad f||")
plt.title("Норма градиента")
plt.legend()
plt.grid()
plt.show()

```



Контур функции

```
x = np.linspace(-2, 2, 400)
y = np.linspace(-1, 3, 400)

X, Y = np.meshgrid(x, y)

Z = a*(X**2 - Y)**2 + b*(X - 1)**2 + f0

plt.figure(figsize=(8,6))

plt.contour(X, Y, Z, levels=50)

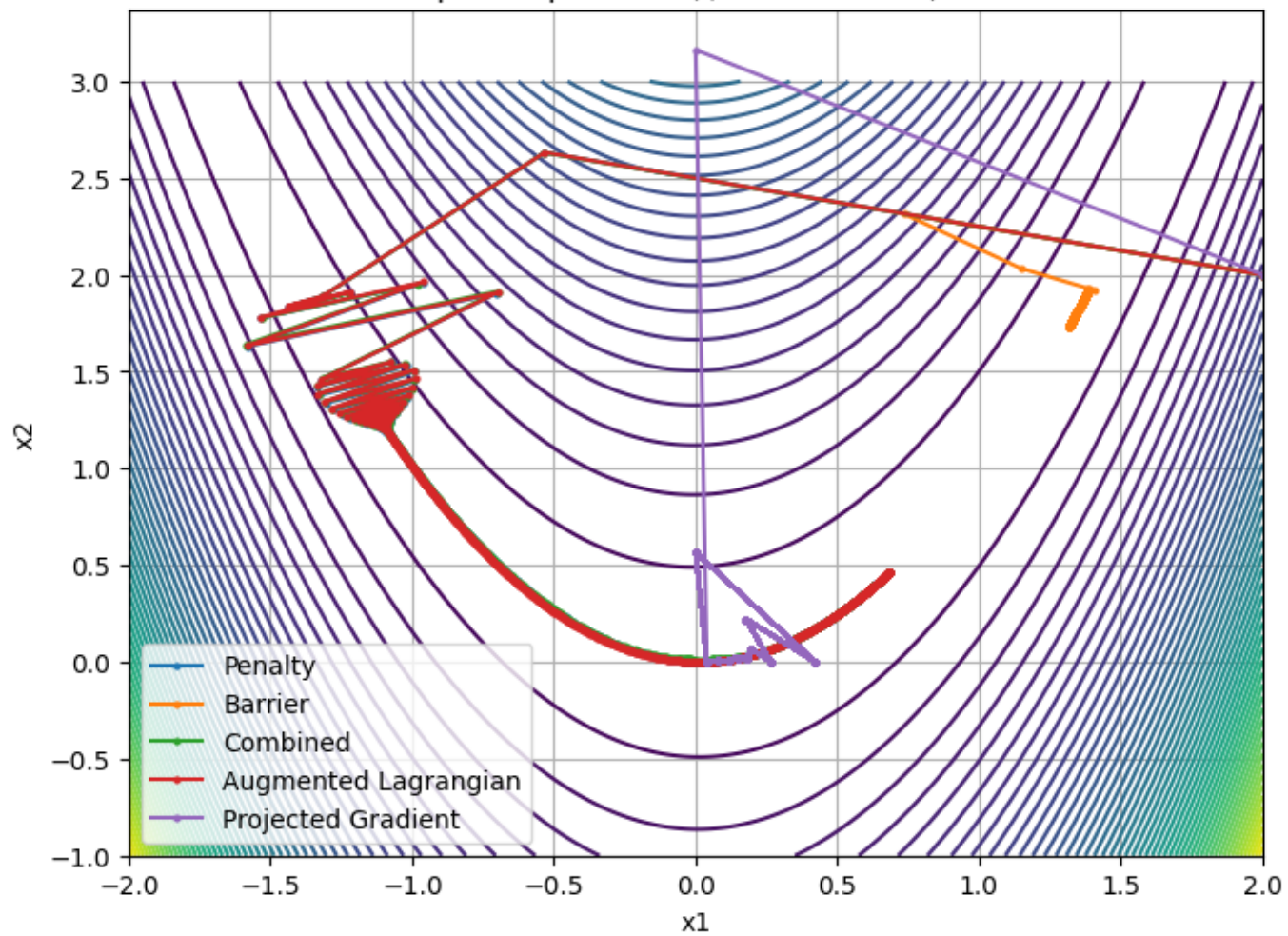
for name, method in methods.items():
    _, history = method(x0)

    path = np.array(history["x"])

    plt.plot(path[:,0], path[:,1], marker='o', markersize=2, label=name)

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Траектории методов оптимизации")
plt.legend()
plt.grid()
plt.show()
```

Траектории методов оптимизации



```
plt.figure(figsize=(9,7))

levels = np.logspace(0, 5, 35)

plt.contour(X, Y, Z, levels=levels)

for name, method in methods.items():
```

```

_, history = method(x0)

path = np.array(history["x"])

# прореживание точек
step = max(1, len(path)//30)

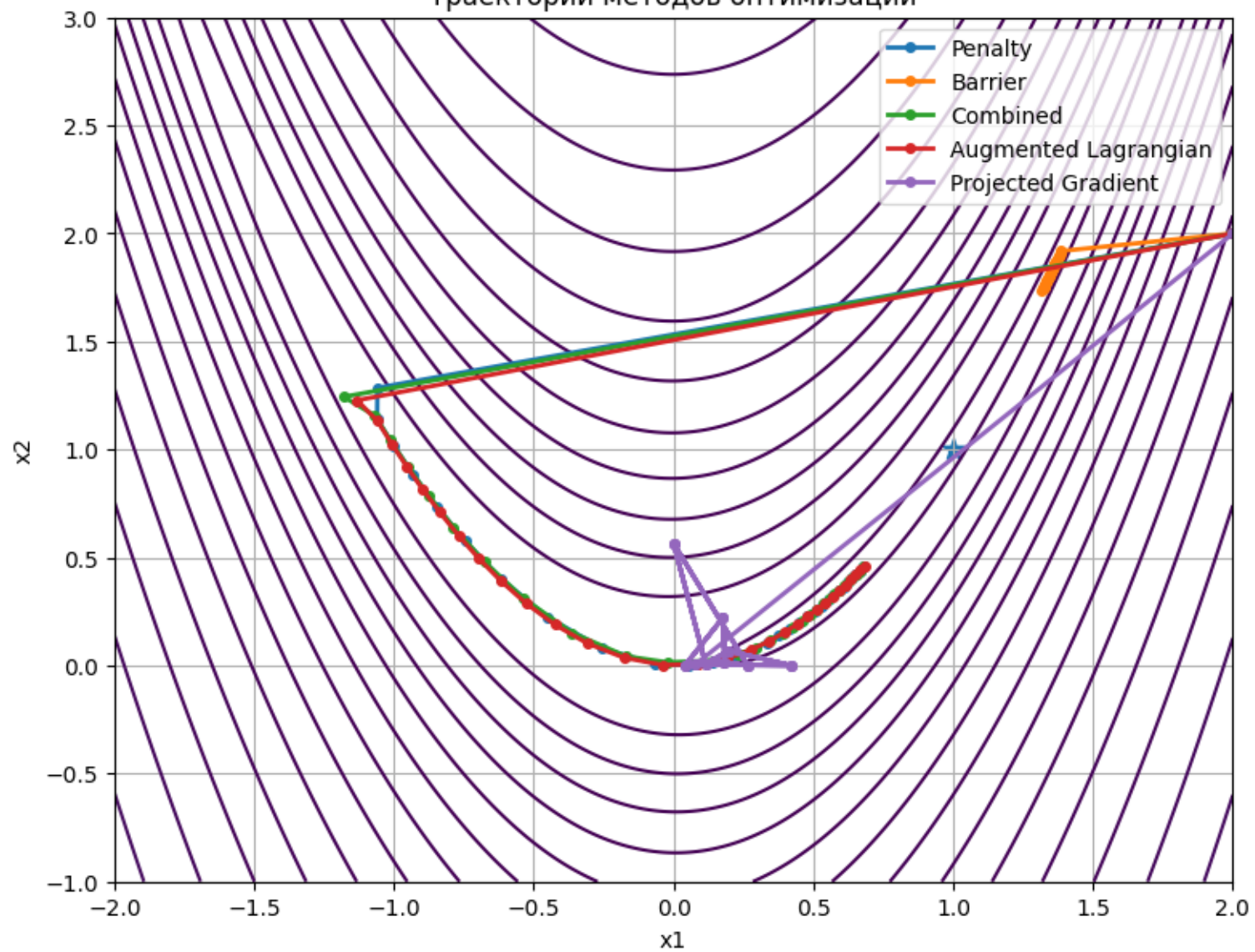
plt.plot(
    path[:,step,0],
    path[:,step,1],
    marker='o',
    markersize=4,
    linewidth=2,
    label=name
)

plt.scatter(1, 1, s=100, marker='*')

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Траектории методов оптимизации")
plt.legend()
plt.grid()
plt.show()

```

Траектории методов оптимизации



Изменение ошибки по мере сходимости

```
plt.figure(figsize=(9,5))

x_star = np.array([1,1])

for name, method in methods.items():

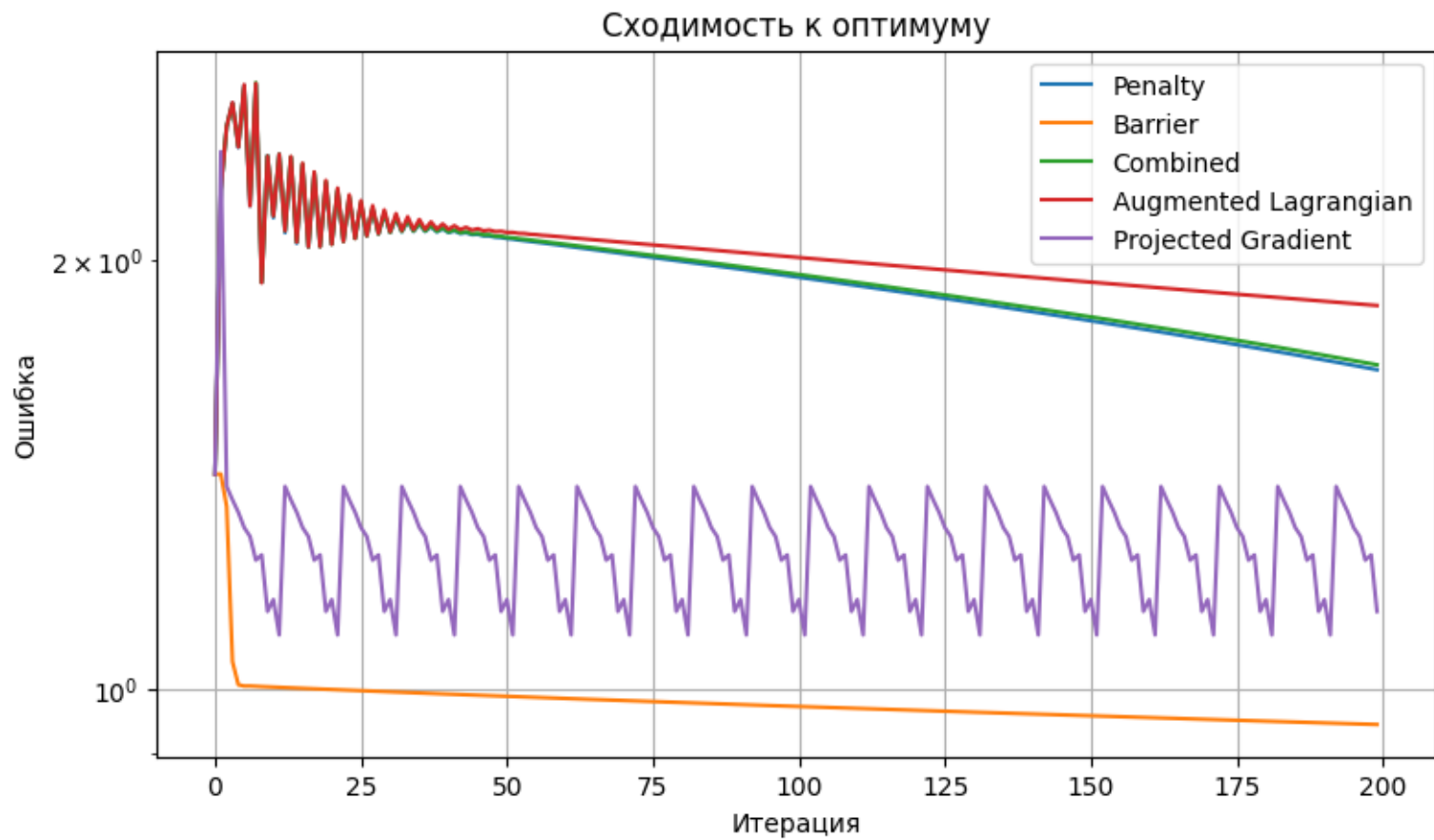
    _, history = method(x0)

    errors = [
        np.linalg.norm(x - x_star)
        for x in history["x"]
    ]

    plt.plot(errors[:200], label=name)

plt.yscale("log")

plt.xlabel("Итерация")
plt.ylabel("Ошибка")
plt.title("Сходимость к оптимуму")
plt.legend()
plt.grid()
plt.show()
```



Сравнение скорости методов

```
import time
import pandas as pd

EPS = 0.2
TARGET = 40

results = []

for name, method in methods.items():
```

```

start_time = time.time()

x_res, history = method(x0)

elapsed = time.time() - start_time

# ищем первую итерацию достижения точности
reached_iter = None

for i, val in enumerate(history["f"]):

    if abs(val - TARGET) <= EPS:
        reached_iter = i
        break

results.append({
    "Метод": name,
    "Минимум x": np.round(x_res, 4),
    "f(x)": round(f(x_res), 6),
    "Итераций до eps": reached_iter,
    "Всего итераций": len(history["f"]),
    "Время (сек)": round(elapsed, 6)
})

df_results = pd.DataFrame(results)

print(df_results)

```

	Метод	Минимум x	f(x)	Итераций до eps	\
0	Penalty	[0.6843, 0.4664]	40.199847	None	
1	Barrier	[1.316, 1.7331]	40.199971	None	
2	Combined	[0.6846, 0.4668]	40.199479	None	
3	Augmented Lagrangian	[0.6845, 0.4666]	40.199687	None	
4	Projected Gradient	[0.4229, 0.0]	45.721797	None	

	Всего итераций	Время (сек)
0	796	0.016539
1	804	0.017408
2	766	0.020763
3	995	0.007604
4	10000	0.112782

Отладочная информация

```
for name, method in methods.items():  
    _, history = method(x0)  
    print(name, len(history["x"]))
```

```
Penalty 796  
Barrier 804  
Combined 766  
Augmented Lagrangian 995  
Projected Gradient 10000
```

```
for name, method in methods.items():  
    _, history = method(x0)  
    print(name)  
    print(history["f"][:10])  
    print()
```

Penalty

```
[np.float64(674.0), np.float64(916.4887913324068), np.float64(54.84088188408072), np.float64(59.94927109734554), np.float64(78.2
```

Barrier

```
[np.float64(674.0), np.float64(674.0), np.float64(539.3953713255033), np.float64(122.03053386577847), np.float64(40.890404237463
```

Combined

```
[np.float64(674.0), np.float64(914.1997465983549), np.float64(54.46141074674354), np.float64(58.94488236326717), np.float64(75.2
```

Augmented Lagrangian

```
[np.float64(674.0), np.float64(916.4887912750079), np.float64(54.70775226301248), np.float64(60.212591190717255), np.float64(78.
```

Projected Gradient

```
[np.float64(674.0), np.float64(1622.0000000000002), np.float64(41.84360448), np.float64(41.70035126720606), np.float64(41.572290
```